# Department of Computer Science and Engineering

| | |
|---|---|
| **Course Code & Name** | CSA06 – Design and Analysis of Algorithms |
| **Assessment** | CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform |
| **Course Outcome Assessed** | Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. |
| **Duration** | 3 Days |
| **Mode** | Individual |
| **Student Reg. No.** | 192525477 |
| **Student Name** | SWATHI K |

**Instructions:** Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute $10^8$ basic operations per second.

## IT APPLICATION-BASED PROBLEM

**Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform:** ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

**Module M1 – Duplicate-Listing Detector:** Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

**Module M2 – API Rate Limiter:** The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

**Module M3 – Product Search:** Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing

the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

**Module M4 – Recommendation Engine:** The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

**Module M5 – Sales Analytics Engine:** The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving T(n) = 2T(n/2) + n log n. Separately, profiling shows that the nightly report generator performs f(n) = 3n² + 10n loǵn + 50 operations.

**(a) Understand the Problem:** For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

**(b) Analyse the Iterative Modules:** For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

**(c) Solve a Recurrence by Substitution:** Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

**(d) Apply the Master Theorem:** For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to T(n) = 2T(n/2) + n log n; if not, derive the complexity using the extended Master Theorem or a recursion tree.

**(e) Analyse Best, Average and Worst Cases:** For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

**(f) Prove an Asymptotic Bound:** Using the formal definitions, prove that the report generator's cost f(n) = 3n² + 10n loǵn + 50 is Θ(n²) by finding constants c̋, ćand n̰, and show that f(n) is not O(n).

**(g) Scalability Decision:** Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

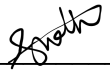
## Rubrics – Application-Based Question

| Criteria | Weightage (%) | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20 | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts | 30 | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20 | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| Accuracy of Solution | 20 | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |

| Criteria | Weightage (%) | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Clarity | 10 | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

**Code of Conduct**

I SWATHI K (192525477) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:  Date: 10/10/26

# A brief overview of the audit and its decisions.

ShopSphere currently stores 100,000 products and expects to grow to 1,000,000 products. This tenfold increase makes algorithmic growth more important than small coding optimisations. The audit examines pairwise duplicate detection, a doubling-window API audit, two product-search approaches, a recommendation recurrence, a sales-analytics recurrence, and a nightly report formula.

The main conclusion is that not every loop with a logarithmic number of rounds costs n log n. M2 processes 1 + 2 + 4 + …, which forms a geometric series and is $\Theta(n)$. Binary search is logarithmic because it halves the remaining search space after each comparison. The current M4 design is quadratic because four recursive subproblems of half size create a total work proportional to $n^2$.

The estimated nightly budget is 600 seconds. Under the simplified operation model, M1 takes about 5,000 seconds, the current M4 design about 10,000 seconds, and the separate report generator about 30,002 seconds. These exceed the budget. M2, binary search, redesigned M4, and M5 are estimated to fit.

| Finding | Result | Action |
|---|---|---|
| M1 duplicate detector | $\Theta(n^2)$, ~83.3 min | Use hash key (seller, normalised title) |
| M2 rate limiter | $\Theta(n)$, 1,048,575 audits | Keep; validate real workload |
| M3 S2 binary search | $\Theta(\log n)$, ~20 comparisons | Use sorted index and iterative form if desired |
| M4 current | $\Theta(n^2)$, ~166.7 min | Remove replicated recursive work |
| M4 redesign | $\Theta(n \log n)$, ~0.20 s in dominant-term model | Preferred recurrence |
| M5 analytics | $\Theta(n \log^2 n)$, ~2.09 s estimate | Monitor constants and actual sort costs |
| Nightly report | $\Theta(n^2)$, ~500 min | Replace repeated quadratic computation |

**Scenario:**

ShopSphere is an online shopping platform. Product records include titles and seller identifiers. Public API requests are audited in windows that double in size. Product search is available through an unsorted sequential scan and a

sorted binary-search index. Recommendation processing and sales analytics use recursive divide-and-conquer algorithms.

## Given data

| Parameter | Value | Meaning |
|---|---|---|
| Current catalogue size | $n = 100{,}000$ | Present scale |
| Target catalogue size | $n = 1{,}000{,}000$ | Expected within a year |
| Server rate | $10^8$ operations/second | Simplified conversion assumption |
| Nightly processing window | 10 minutes = 600 seconds | Maximum allowed estimated time |
| M1 operation | Product-pair comparison | Checks title and seller |
| M2 operation | Audit(Request[j]) | Counts one request audit |
| S2 recurrence | $T(n)=T(n/2)+1$, $T(1)=1$ | One comparison per level |
| M4 current | $4T(n/2)+n$ | Replicated partitions |
| M4 redesign | $2T(n/2)+n$ | Replication removed |
| M5 | $2T(n/2)+n \log_2 n$ | Sorting/processing work per level |
| Report function | $3n^2+10n \log_2 n+50$ | Profiled operation count |

## Required deliverables

- Identify input size and dominant basic operation for each module.
- Derive M1 and M2 counts, including the $n=1{,}024$ verification.
- Solve S2 using backward substitution and discuss space complexity.
- Apply the Master Theorem to M4 and analyse M5 with an extended case or recursion tree.
- Compare best, average, and worst search cases; evaluate the probability model for S1.
- Prove the report generator is $\Theta(n^2)$ and not $O(n)$.
- Estimate operations and time at $n=1{,}000{,}000$, then recommend redesigns.

The analysis uses the standard RAM-style model: each stated basic operation is counted as one operation, and the server is assumed to execute 100,000,000

such operations each second. The time estimate is therefore Tseconds = operation count / $10^8$.

This is a comparison model rather than a benchmark. It excludes network latency, database disk access, cache misses, thread scheduling, memory allocation, distributed communication, and differences in the cost of string comparison versus integer arithmetic. Real systems must be profiled after implementation.

## Asymptotic notation

| Notation | Meaning | How it is used here |
|---|---|---|
| O(g(n)) | Asymptotic upper bound | Shows a function does not grow faster than a constant multiple of g(n), eventually |
| Ω(g(n)) | Asymptotic lower bound | Shows a function grows at least as fast as a constant multiple of g(n), eventually |
| Θ(g(n)) | Tight asymptotic bound | Upper and lower bounds match up to constant factors |
| o(g(n)) | Strictly smaller growth | Useful in Master Theorem Case 1 conditions |
| ω(g(n)) | Strictly larger growth | Useful for comparing dominant terms |

## Important assumptions

- For the exact M2 formula, n is a positive integer and the loop uses i = 1, 2, 4, … with condition i < n.
- For the S2 substitution derivation, n is initially treated as a power of two; ceilings handle other sizes.
- The Master Theorem analyses assume a consistent base case and a recurrence that represents the work.
- Operation counts in part (g) are dominant-term estimates where exact constants are not specifiedThe input size n is the number of product, request, customer–product, or sales records being processed. For a recurrence, n is the size of the data passed to the function. The basic operation is the repeated operation most directly responsible for the growth of the running time.

| Module | Input size | Dominant operation | Where it occurs | Expected growth |
|---|---|---|---|---|
| M1 | n products | Compare two products' title and seller | Inside both nested loops | $\Theta(n^2)$ |
| M2 | n request positions | Audit one request | Inside inner loop for each doubling value | $\Theta(n)$ |
| M3 S1 | n catalogue entries | Compare target with current entry | Sequential scan | Best $\Theta(1)$, worst $\Theta(n)$ |
| M3 S2 | n sorted entries | Compare target with middle entry | At each halving step | $\Theta(\log n)$ |
| M4 current | n data items | Recursive work and merge | Four calls on n/2 plus n work | $\Theta(n^2)$ |
| M4 redesign | n data items | Recursive work and merge | Two calls on n/2 plus n work | $\Theta(n \log n)$ |
| M5 | n sales records | Sort/merge processing | Two calls plus n log n work per call | $\Theta(n \log^2 n)$ |

**Justification:**

nested loops over pairs lead to a triangular number. A doubling loop sums powers of two rather than performing n operations in each round. Sequential search may inspect every item. Binary search halves the remaining candidates. Recursive algorithms are classified by the number and size of subproblems and the non-recursive work done at each call.

The M1 routine compares every product with every later product. It avoids comparing an item with itself and avoids repeating the same unordered pair in reverse order.

Pseudocode

```
FOR i ← 0 TO n − 2
    FOR j ← i + 1 TO n − 1
        IF Product[i].title = Product[j].title AND
```

```
        Product[i].seller = Product[j].seller
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

**Counting comparisons by outer-loop index**

When i = 0, j takes n−1 values. When i = 1, j takes n−2 values. The final outer-loop value i = n−2 produces one comparison. The total number of pairs is therefore the sum of integers from 1 to n−1.

| i value | Number of j values | Running total |
|---|---|---|
| 0 | n−1 | n−1 |
| 1 | n−2 | 2n−3 |
| 2 | n−3 | 3n−6 |
| … | … | … |
| n−2 | 1 | n(n−1)/2 |

The IF statement may perform two field comparisons for a pair, but the algorithm's pair count is the main repeated operation. If title comparison length varies, real runtime can be higher than this unit-cost estimate.

**Exact comparison count**

$C(n) = (n-1) + (n-2) + \ldots + 2 + 1$.

Reverse the same sum: $C(n) = 1 + 2 + 3 + \ldots + (n-1)$.

Add the two expressions term by term: $2C(n) = n + n + \ldots + n$, with $n-1$ copies of n.

Therefore $2C(n) = n(n-1)$, and $C(n) = n(n-1)/2$.

Expanded form: $C(n) = (n^2-n)/2 = 0.5n^2-0.5n$.

**Asymptotic result**

The $n^2$ term dominates the linear term as n grows. Thus $C(n) = \Theta(n^2)$. The upper and lower bounds are both quadratic.

**Space complexity:**
The two loops use only the integer variables i and j. If duplicate marks are written directly into existing records and no extra list of all pairs is created, auxiliary space is $\Theta(1)$. The product catalogue itself is input storage and is not counted as auxiliary space.

At n = 100,000
C(100,000) = 100,000 × 99,999 / 2 = 4,999,950,000 comparisons. At $10^8$ operations/second, the simplified estimate is 49.9995 seconds.

At n = 1,000,000
C(1,000,000) = 1,000,000 × 999,999 / 2 = 499,999,500,000 comparisons. Estimated time = 499,999,500,000 / 100,000,000 = 4,999.995 seconds, approximately 83.33 minutes.

**Conclusion:** M1 exceeds the 10-minute window at the target catalogue size.

A hash table can avoid comparing every product with every other product. Build a composite key from the seller identifier and a normalised title. For each product, check whether the key already exists in the table; if it exists, mark the listing as a duplicate. Otherwise, insert the key.

**Suggested high-level procedure**

1. Create an empty hash set H.
2. For each product, normalise the title using a consistent rule.
3. Create key = (seller_id, normalised_title).
4. If key is in H, mark this product as a duplicate.
5. Otherwise insert key into H.
6. Continue until all n products are processed.

## Complexity comparison:

| Approach | Expected time | Auxiliary space | Main trade-off |
|---|---|---|---|
| Pairwise comparison | $\Theta(n^2)$ | $\Theta(1)$ beyond records | Simple but too slow at scale |
| Hash-based detection | Expected $\Theta(n)$ | $\Theta(n)$ | Uses memory; hash collisions must be handled |
| Sort by composite key | $\Theta(n \log n)$ | Depends on sort | Predictable comparison-based alternative |

The hash-based option is recommended if expected-time performance and memory availability are acceptable. A sort-based approach is an alternative when predictable worst-case behaviour or ordered duplicate groups are useful. Title normalisation rules must be agreed: case folding, whitespace, punctuation, and Unicode treatment can affect whether two titles count as equal.

M2 starts at i = 1. The inner loop audits i requests, then i doubles. The outer loop does not perform n audits on every iteration; the inner loop length changes geometrically.

i = 1 → 1 audit

i = 2 → 2 audits

i = 4 → 4 audits

i = 8 → 8 audits

The loop stops before the first i value that is greater than or equal to n.

### Why Θ(n log n) is not correct

The team member counted approximately $\log_2 n$ outer-loop rounds and multiplied that count by n, as if each round processed n requests. But the inner loop processes 1, 2, 4, 8, and so on. The sum is dominated by the largest power of two and is linear, not n log n.

**Geometric series identity**

For $k \geq 1$, $1 + 2 + 4 + \ldots + 2^{(k-1)} = 2^k - 1$.

This identity is the key to deriving the exact operation count.

Let $k = \text{ceil}(\log_2 n)$, for $n > 1$. The powers used by the loop are $2^0, 2^1, \ldots, 2^{(k-1)}$. Their sum is:

$A(n) = 1 + 2 + 4 + \ldots + 2^{(k-1)}$

$A(n) = (2^k - 1)/(2 - 1) = 2^k - 1$

Therefore $A(n) = 2^{\text{ceil}(\log_2 n)} - 1$ for $n > 1$. For $n \leq 1$, the loop performs zero audits.

**Tight asymptotic bound**

By the ceiling definition, $n \leq 2^{\text{ceil}(\log_2 n)} < 2n$. Subtracting 1 gives $n-1 \leq A(n) < 2n-1$. For sufficiently large n, these are linear upper and lower bounds. Thus $A(n) = \Theta(n)$.

Verification for n = 1,024

$1{,}024 = 2^{10}$. Since the loop condition is $i < 1{,}024$, the largest used value is $512 = 2^9$.

$A(1{,}024) = 1 + 2 + 4 + \ldots + 512 = 2^{10} - 1 = 1{,}023$ audits.

For n = 1,000,000

$\text{ceil}(\log_2 1{,}000{,}000) = 20$, so $A(n) = 2^{20} - 1 = 1{,}048{,}575$ audits. Estimated time $= 1{,}048{,}575 / 10^8 = 0.01048575$ seconds.

**Conclusion:** the claim $\Theta(n \log n)$ is incorrect; the tight bound is $\Theta(n)$.

ShopSphere has two search versions. S1 scans entries in sequence and works even when the catalogue is unsorted. S2 compares the target against the middle item in a sorted index and discards half the remaining range after each comparison.

| Feature | S1: Sequential search | S2: Binary search |
|---|---|---|
| Data order | Can be unsorted | Must be sorted by search key |
| Best case | 1 comparison | 1 comparison |
| Worst-case growth | $\Theta(n)$ | $\Theta(\log n)$ |
| Average growth | $\Theta(n)$ for uniform positions | $\Theta(\log n)$ |
| Iterative auxiliary space | $\Theta(1)$ | $\Theta(1)$ |
| Recursive auxiliary space | Not needed for basic scan | $\Theta(\log n)$ call stack |
| Use case | Small/unsorted lists | Large sorted index |

**Basic operation**

For both algorithms, the basic operation is comparing the target product key with the key of a catalogue entry. For S1, the comparison repeats along the list. For S2, each comparison reduces the remaining candidate range by roughly half.

**Practical condition**

Binary search is not a direct replacement for sequential search unless the index is sorted and the sort/index maintenance cost is acceptable. E-commerce systems often maintain a sorted or indexed search structure so that lookup does not require sorting the full catalogue for every request.

Given recurrence: $T(n) = T(n/2) + 1$, with $T(1) = 1$. Assume n is a power of two so the division reaches exactly 1.

Step 1: $T(n) = T(n/2) + 1$

Step 2: Substitute $T(n/2) = T(n/4) + 1$.

$T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$

Step 3: Substitute again.

$T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3$

Step 4: After k substitutions:

$T(n) = T(n/2^k) + k$

Step 5: Reach the base case when $n/2^k = 1$.

$n/2^k = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$

Step 6: Substitute the stopping depth into the expression.

$T(n) = T(1) + \log_2 n = 1 + \log_2 n$

Closed form and complexity
For powers of two, the exact closed form under the given base case is $T(n) = \log_2 n + 1$. Therefore $T(n) = \Theta(\log n)$. For non-powers of two, the number of halving levels is bounded by a ceiling, so the asymptotic result remains $\Theta(\log n)$.

**Iterative binary search:**
The iterative implementation stores low, high, mid, and perhaps the target key. The number of variables does not increase with n. Auxiliary space is $\Theta(1)$.

**Recursive binary search:**
The recursive version calls itself once for the remaining half. Its recursion depth is logarithmic, so the call stack needs $\Theta(\log n)$ space. A recursive implementation can be clear but has stack overhead.

**Comparison estimate for one million entries:**
$\log_2(1{,}000{,}000) \approx 19.9316$. The recurrence gives about $1 + \log_2 n \approx 20.9316$ in a continuous estimate; actual comparison counts are integers and depend on the exact tree shape and stopping rule. A common worst-case upper estimate is $\text{ceil}(\log_2(n+1)) = 20$ comparisons for the standard binary-search comparison model, though variants and how equality/range checks are counted can shift the exact count by one.

For this report's scalability table, approximately 20 key comparisons are used as a simple order-of-growth estimate. At $10^8$ operations per second, this theoretical count is negligible; real request latency will be dominated by indexing, memory, network, and service overhead rather than twenty comparisons.

**Pseudocode:**

low = 0; high = n − 1

WHILE low ≤ high:

    mid = floor((low + high) / 2)

    compare target with A[mid]

    if equal: return mid

if target < A[mid]: high = mid − 1    else: low = mid + 1

```python
import math

OPS_PER_SECOND = 10**8

def m1_comparisons(n):
    return n * (n - 1) // 2

def m2_audits(n):
    # i = 1, 2, 4, ... while i < n
    return (1 << math.ceil(math.log2(n))) - 1 if n > 1 else 0

def binary_search_comparisons(n):
    # Approximate worst-case comparisons for a sorted index
    return math.ceil(math.log2(n + 1))

def m4_current(n):
    # T(n) = 4T(n/2) + n -> dominant term is n^2
    return n * n

def m4_redesign(n):
    # T(n) = 2T(n/2) + n -> estimate n log2(n)
    return round(n * math.log2(n))

def m5_estimate(n):
    # Recursion-tree sum: n * L(L+1)/2, L = log2(n)
    L = math.log2(n)
```

```
31
32 def show(name, operations):
33     seconds = operations / OPS_PER_SECOND
34     print(f"{name}")
35     print(f"  Estimated operations: {operations:,}")
36     print(f"  Estimated time: {seconds:,.4f} seconds")
37     print()
38
39 def main():
40     n = 1_000_000
41     print("SHOPSPHERE SCALABILITY AUDIT")
42     print(f"Catalogue size n = {n:,}")
43     print(f"Server speed = {OPS_PER_SECOND:,} operations/second\n")
44
45     show("M1 - Duplicate-listing detector", m1_comparisons(n))
46     show("M2 - API rate limiter", m2_audits(n))
47     show("M3 S2 - Binary search (worst-case estimate)",
48          binary_search_comparisons(n))
49     show("M4 - Current recommendation engine", m4_current(n))
50     show("M4 - Redesigned recommendation engine", m4_redesign(n))
51     show("M5 - Sales analytics (recursion-tree estimate)", m5_estimate(n))
52     show("Nightly report generator", report_generator(n))
53
54     print("M2 verification for n = 1,024:")
55     print(f"  Audit operations = {m2_audits(1024):,} (expected 1,023)")
56
```

| Case | Input condition | Comparisons |
|---|---|---|
| Best | Target is at the first position | 1 |
| Average successful | Target exists; each of n positions equally likely | (n+1)/2 |
| Worst successful | Target is at the last position | n |
| Worst unsuccessful | Target is absent; all entries are inspected | n |

**Average when target may be absent:**

Let p be the probability that the target exists. If it exists, assume its position is uniformly distributed, giving average (n+1)/2 comparisons. If it does not exist, the search examines all n entries. By the law of total expectation:

$E[C] = p \times (n+1)/2 + (1-p) \times n$

For p = 0.5 and n = 1,000,000:

$E[C] = 0.5 \times 1{,}000{,}001/2 + 0.5 \times 1{,}000{,}000$

$E[C] = 250{,}000.25 + 500{,}000 = 750{,}000.25$ comparisons.

**Interpretation:** at a million products, an average S1 query may need roughly three-quarters of a million comparisons under these assumptions. This is too expensive for frequent interactive search compared with S2's logarithmic number of comparisons.

| Case | Input condition | Comparison count |
|---|---|---|
| Best | Target equals the first middle element | 1 |
| Average | Target lies at a typical depth in the balanced search tree | $\Theta(\log n)$ |
| Worst | Target is found only after the deepest required halving steps, or is absent | $\Theta(\log n)$ |

**Why all cases differ from S1:**
In sequential search, the target's location determines how many earlier elements must be checked. In binary search, every comparison removes about half of the remaining candidates. Even an unfavourable location requires only logarithmically many halvings.

For n = 1,000,000
The logarithm base 2 is approximately 19.93. Therefore best case is one comparison and the average/worst cases are around twenty comparisons, depending on exact implementation and whether loop-condition comparisons are counted.

**Trade-offs:**

- S2 requires sorted data or a maintained sorted index.
- Updates to the catalogue may require index maintenance, which adds work outside the lookup operation.
- S1 is useful for very small collections or one-off scans where index construction is not worthwhile.
- For a growing e-commerce catalogue, a maintained index usually makes logarithmic search preferable for repeated queries.

Current recurrence: $T(n) = 4T(n/2) + n$.

**Identify the parameters:**

| Parameter | Value | Explanation |
|---|---|---|
| a | 4 | Number of recursive subproblems |
| b | 2 | Each subproblem has size n/2 |
| f(n) | n | Linear merge/non-recursive work |
| n^(log_b a) | $n^{(\log_2 4)} = n^2$ | Critical comparison function |

**Select the case:**

Compare f(n) = n with $n^{(\log_b a)} = n^2$. Since $n = O(n^{(2-\varepsilon)})$ for $\varepsilon = 1$, f(n) is polynomially smaller than $n^2$. This is Master Theorem Case 1.

Therefore $T(n) = \Theta(n^{(\log_b a)}) = \Theta(n^2)$.

**Recursion-tree intuition:**

At level 0, non-recursive work is n. At level 1, there are four subproblems of size n/2, so total merge work is $4 \times (n/2) = 2n$. At level 2, there are 16 subproblems of size n/4, giving total work $16 \times (n/4) = 4n$. Each level's work doubles. The number of levels is $\log_2 n$, and the leaves total $n^2$ base-case contributions up to constants. The bottom levels dominate, producing $\Theta(n^2)$.

**Proposed recurrence:** $T(n) = 2T(n/2) + n$. The redesign removes replication, reducing the number of recursive subproblems from four to two.

| Parameter | Value | Explanation |
|---|---|---|
| a | 2 | Two recursive subproblems |
| b | 2 | Each has size n/2 |
| f(n) | n | Linear merge/non-recursive work |
| n^(log_b a) | $n^{(\log_2 2)} = n$ | Critical comparison function |

**Select the case:**

Here $f(n) = \Theta(n^{(\log_b a)}) = \Theta(n)$. This is Master Theorem Case 2 with k = 0, so $T(n) = \Theta(n \log n)$.

**Recursion-tree intuition:**

At every level, there are $2^i$ subproblems of size $n/2^i$. The total linear merge work on that level is $2^i \times n/2^i = n$. Each level costs n, and there are $\log_2 n$ levels. The total is $n \log_2 n$.

**Effect of doubling data:**

For a quadratic function, doubling n multiplies the dominant work by about four. For $n \log_2 n$, the ratio is $2n \log_2(2n)/(n \log_2 n) = 2(\log_2 n + 1)/\log_2 n$. At $n = 1{,}000{,}000$ this is about 2.1003. The redesign scales far better than the current design.

| Measure | Current M4 | Redesigned M4 |
|---|---|---|
| Recurrence | $4T(n/2)+n$ | $2T(n/2)+n$ |
| a, b | 4, 2 | 2, 2 |
| Critical term | $n^2$ | n |
| Master case | Case 1 | Case 2 |
| Tight complexity | $\Theta(n^2)$ | $\Theta(n \log n)$ |
| Dominant estimate at $n=10^6$ | $10^{12}$ operations | ~19.93 million operations |
| Estimated time at $10^8$ ops/s | 10,000 s ≈ 166.7 min | ~0.199 s |
| 10-minute requirement | Fail | Pass in simplified model |

**Recommendation:**

The redesign is strongly recommended because it changes the asymptotic growth class from quadratic to n log n. The estimated ratio in dominant work is approximately $n/\log_2 n$, which is around 50,000 at $n = 1{,}000{,}000$. The exact ratio in a real application depends on constants, base cases, data movement, and the details of the replication strategy.

Before deployment, the team should confirm that removing replication does not violate correctness, availability, fault tolerance, or data consistency requirements. Algorithmic efficiency must be evaluated together with system-level requirements

**Sales analytics recurrence:** $T(n) = 2T(n/2) + n \log_2 n$.

| Parameter | Value |
|---|---|
| a | 2 |
| b | 2 |
| f(n) | $n \log_2 n$ |
| n^(log_b a) | $n^{(\log_2 2)} = n$ |

The simplest three-case version is often introduced for f(n) that is polynomially smaller than, equal to, or polynomially larger than n^(log_b a). Here f(n) is n multiplied by a logarithmic factor. It is not polynomially smaller, and it is not simply Θ(n) because of the extra log n factor.

The extended Master Theorem Case 2 handles $f(n) = \Theta(n^{(\log_b a)} \log^k n)$. Here k = 1, so $T(n) = \Theta(n^{(\log_b a)} \log^{(k+1)} n) = \Theta(n \log^2 n)$.

**Final result:**

$T(n) = \Theta(n \log^2 n)$. This is worse than n log n by a logarithmic factor, but it is still asymptotically much better than $n^2$.

Let $L = \log_2 n$. At the root, the non-recursive work is $n \log_2 n$.

At level 1, there are 2 subproblems of size n/2. Each contributes $(n/2) \log_2(n/2)$, so total work is $2 \times (n/2) \times (L-1) = n(L-1)$.

At level i, there are 2^i subproblems of size n/2^i. Each subproblem contributes $(n/2^i)(L-i)$, so the total level cost is:

$W_i = 2^i \times (n/2^i) \times (L-i) = n(L-i)$.

For levels i = 0 through L−1, total internal work is:

$W = n[L + (L-1) + (L-2) + \ldots + 1]$

$W = n \times L(L+1)/2$

Since $L = \log_2 n$, $W = \Theta(n \log^2 n)$. Leaf work is Θ(n) for constant work per leaf and does not change the dominant result.

At n = 1,000,000
L ≈ 19.9316. The estimate $nL(L+1)/2$ is about 208,599,497 operations. At $10^8$ operations per second, this is approximately 2.086 seconds.

This estimate treats the recurrence's non-recursive work as exactly $n \log_2 n$ and uses a continuous logarithm. A practical implementation's constants and sorting details may change the measured runtime.

| Growth class | Example from audit | Practical meaning |
|---|---|---|
| $\Theta(1)$ | Constant-space loop variables; one best-case search comparison | Work does not increase with n for the counted operation |
| $\Theta(\log n)$ | S2 binary search | A tenfold increase adds only a few comparisons |
| $\Theta(n)$ | M2 audits; expected hash scan | Work grows approximately in proportion to input size |
| $\Theta(n \log n)$ | Redesigned M4 | Slightly more than linear growth |
| $\Theta(n \log^2 n)$ | M5 sales analytics | One extra logarithmic factor over n log n |
| $\Theta(n^2)$ | M1 pairwise detector; current M4; report function | Tenfold growth can mean roughly 100 times the dominant work |

Growth from 100,000 to 1,000,000
The input grows by a factor of 10. A linear algorithm grows by about 10 times; n log n grows by slightly more than 10 times; $n \log^2 n$ grows by a little more; a quadratic algorithm grows by about 100 times. These ratios explain why the current quadratic modules become the main bottleneck.

Asymptotic analysis does not predict every exact second, but it is highly useful for choosing algorithms before the system reaches the larger input size

Given $f(n) = 3n^2 + 10n \log_2 n + 50$. By definition, $f(n)$ is $\Theta(n^2)$ if there exist positive constants $c_1$, $c_2$ and $n_0$ such that $c_1 n^2 \leq f(n) \leq c_2 n^2$ for every $n \geq n_0$.

**Upper-bound proof:**
For $n \geq 2$, $\log_2 n \leq n$. Multiplying both sides by 10n gives:

$10n \log_2 n \leq 10n^2$.

Also, for $n \geq 2$, $50 \leq 12.5n^2$, because $n^2 \geq 4$.

Therefore:

$f(n) = 3n^2 + 10n \log_2 n + 50$

$f(n) \leq 3n^2 + 10n^2 + 12.5n^2$

$f(n) \leq 25.5n^2 \leq 26n^2$.

Thus $c_2 = 26$ is a valid upper-bound constant for $n \geq 2$

**Lower-bound proof:**

All terms in f(n) are non-negative for $n \geq 1$. Therefore:

$f(n) = 3n^2 + 10n \log_2 n + 50 \geq 3n^2$.

Choose $c_1 = 3$.

**Combine the bounds:**

For every $n \geq 2$:

$3n^2 \leq 3n^2 + 10n \log_2 n + 50 \leq 26n^2$.

This satisfies the formal definition with $c_1 = 3$, $c_2 = 26$ and $n_0 = 2$.

Therefore $f(n) = \Theta(n^2)$.

Why the lower-order terms do not change the class

The term $10n \log_2 n$ grows more slowly than $n^2$, and the constant 50 does not depend on n. As n becomes large, $3n^2$ dominates the expression. The formal constants prove this without relying only on intuition.

Assume for contradiction that $f(n) = O(n)$. By the definition of Big-O, there would be constants $c > 0$ and $n_1$ such that $f(n) \leq cn$ for all $n \geq n_1$.

But $f(n) \geq 3n^2$. Combining the inequalities gives:

$3n^2 \leq cn$.

For $n > 0$, divide both sides by n:

$3n \leq c$.

The constant c is fixed, but 3n becomes arbitrarily large as n increases. Therefore $3n \leq c$ cannot hold for every sufficiently large n.

This contradiction shows that f(n) is not O(n).

**Interpretation**

The report generator cannot be described as linear-time because its leading term is quadratic. Even though it also contains n log n and constant terms, those terms do not remove the quadratic cost.

The result matters for planning: a system that repeatedly performs this report computation for every product can exceed a strict nightly deadline when the catalogue becomes large.

Use server speed $R = 10^8$ operations/second and budget $B = 600$ seconds. For an estimated operation count C, estimated time is C/R. A module exceeds the budget when $C > 600 \times 10^8 = 60{,}000{,}000{,}000$ operations.

| Module | Operation estimate | Time estimate | Budget status |
|---|---|---|---|
| M1 | 499,999,500,000 | 4,999.995 s ≈ 83.33 min | FAIL |
| M2 | 1,048,575 | 0.01049 s | PASS |
| M3 S2 | about 20 comparisons | about 0.0000002 s | PASS |
| M4 current | about 1,000,000,000,000 | 10,000 s ≈ 166.67 min | FAIL |
| M4 redesign | about 19,931,569 | 0.1993 s | PASS |
| M5 | about 208,599,497 | 2.0860 s | PASS |
| Nightly report | 3,000,199,315,736 | 30,001.993 s ≈ 500.03 min | FAIL |

The estimates show three clear failures: M1, current M4, and the separate nightly report generator. These values are based on the stated operation model and dominant-term assumptions; they should be validated against actual code and hardware.

| Priority | Module | Proposed change | Reason |
|---|---|---|---|
| 1 | Nightly report generator | Replace repeated quadratic calculation with indexed aggregation, incremental updates, or a one-pass summary where logically possible | ~500 minutes exceeds budget |
| 2 | M1 duplicate detector | Use hash set keyed by seller and normalised title; alternatively sort by composite key | $\Theta(n^2)$ pair checks exceed budget |
| 3 | M4 current | Remove duplicated replicated recursion or | Changes $\Theta(n^2)$ to $\Theta(n$ |

| | | | |
|---|---|---|---|
| | recommendation engine | share intermediate results; use the two-subproblem design if correctness permits | log n) |
| 4 | M2 rate limiter | Retain geometric loop; add correctness tests for boundary values and real request semantics | $\Theta(n)$ is efficient |
| 5 | M3 S2 | Maintain a sorted index and prefer iterative binary search if stack simplicity is desired | $\Theta(\log n)$ comparisons |
| 6 | M5 analytics | Retain if measurements confirm the estimate; optimise sorting and data movement if needed | $\Theta(n \log^2 n)$ fits the theoretical budget |

## Safe rollout

7. Build unit tests for empty, one-item, duplicate, absent-target, and boundary-size inputs.
8. Compare old and new outputs on the same representative data.
9. Measure wall-clock time and peak memory with production-like data.
10. Deploy the redesigned module behind a feature flag.
11. Monitor error rates, duplicate-detection accuracy, and nightly completion time.
12. Keep a rollback plan until results are stable.

The following Python program is an analytical estimator for the assignment. It computes formula-based operation counts for the modules at n = 1,000,000 and prints estimated times using $10^8$ operations per second. It does not load real product records or implement a production database.

## Functions in the code:

| Function | Purpose | Formula |
|---|---|---|
| m1_comparisons(n) | Pairwise comparisons | $n(n-1)/2$ |
| m2_audits(n) | Doubling-loop audits | 2^ceil($\log_2 n$)−1 |
| binary_search_comparisons(n) | Comparison estimate | ceil($\log_2(n+1)$) |
| m4_current(n) | Dominant estimate | $n^2$ |
| m4_redesign(n) | Dominant estimate | $n \log_2 n$ |
| m5_estimate(n) | Recursion-tree estimate | $nL(L+1)/2$, $L=\log_2 n$ |
| report_generator(n) | Profiled formula | $3n^2+10n\log_2 n+50$ |
| show(name, operations) | Converts count to seconds | operations / $10^8$ |

# Figure 1. Screenshot-style image of the Python implementation used for

shopsphere_scalability_audit.py — Code Screenshot

```
import math

OPS_PER_SECOND = 10**8

def m1_comparisons(n):
    return n * (n - 1) // 2

def m2_audits(n):
    # i = 1, 2, 4, ... while i < n
    if n <= 1:
        return 0
    return (1 << math.ceil(math.log2(n))) - 1

def binary_search_comparisons(n):
    # Worst-case upper estimate for a sorted index
    return math.ceil(math.log2(n + 1))

def m4_current(n):
    # Dominant-term estimate for T(n) = 4T(n/2) + n
    return n * n

def m4_redesign(n):
    # Dominant-term estimate for T(n) = 2T(n/2) + n
    return round(n * math.log2(n))

def m5_estimate(n):
    # Recursion-tree estimate for T(n) = 2T(n/2) + n log2(n)
    L = math.log2(n)
    return round(n * L * (L + 1) / 2)

def report_generator(n):
    return round(3*n*n + 10*n*math.log2(n) + 50)

def show(name, operations):
    seconds = operations / OPS_PER_SECOND
    print(name)
    print(f"  Estimated operations: {operations:,}")
    print(f"  Estimated time: {seconds:,.4f} seconds")
    print()

def main():
    n = 1_000_000
    print("SHOPSPHERE SCALABILITY AUDIT")
    print(f"Catalogue size n = {n:,}")
    print(f"Server speed = {OPS_PER_SECOND:,} operations/second")
    print()

    show("M1 - Duplicate-listing detector", m1_comparisons(n))
    show("M2 - API rate limiter", m2_audits(n))
    show("M3 S2 - Binary search (worst-case estimate)",
         binary_search_comparisons(n))
    show("M4 - Current recommendation engine", m4_current(n))
    show("M4 - Redesigned recommendation engine", m4_redesign(n))
    show("M5 - Sales analytics (recursion-tree estimate)", m5_estimate(n))
    show("Nightly report generator", report_generator(n))

    print("M2 verification for n = 1,024:")
    print(f"  Audit operations = {m2_audits(1024):,} (expected 1,023)")

if __name__ == "__main__":
    main()
```

**Important limitation**

The program estimates operation counts from the assignment's mathematical models. It should not be presented as a benchmark of actual e-commerce server performance. Actual profiling is required before deployment decisions.

## Figure 2. Output from running the Python estimator.

Python Run — Expected Output Screenshot

```
SHOPSPHERE SCALABILITY AUDIT
Catalogue size n = 1,000,000
Server speed = 100,000,000 operations/second

M1 - Duplicate-listing detector
  Estimated operations: 499,999,500,000
  Estimated time: 4,999.9950 seconds

M2 - API rate limiter
  Estimated operations: 1,048,575
  Estimated time: 0.0105 seconds

M3 S2 - Binary search (worst-case estimate)
  Estimated operations: 20
  Estimated time: 0.0000 seconds

M4 - Current recommendation engine
  Estimated operations: 1,000,000,000,000
  Estimated time: 10,000.0000 seconds

M4 - Redesigned recommendation engine
  Estimated operations: 19,931,569
  Estimated time: 0.1993 seconds

M5 - Sales analytics (recursion-tree estimate)
  Estimated operations: 208,599,497
  Estimated time: 2.0860 seconds

Nightly report generator
  Estimated operations: 3,000,199,315,736
  Estimated time: 30,001.9932 seconds

M2 verification for n = 1,024:
  Audit operations = 1,023 (expected 1,023)
```

**How to interpret the output**

- M1 has roughly 500 billion comparisons, giving about 83.3 minutes in the stated model.
- M2 has 1,048,575 audits for one million requests and about 0.0105 seconds under the same model.
- M4 current is quadratic and exceeds the 10-minute window; redesigned M4 is n log n.
- M5 is estimated at about 208.6 million operations, or about 2.09 seconds.
- The report generator is approximately 3.0002 trillion operations and takes about 500 minutes.

M2 is also verified for n = 1,024: the sum 1 + 2 + … + 512 equals 1,023 audits.

## Conclusion

The ShopSphere audit demonstrates how asymptotic analysis predicts the impact of catalogue growth. M1's pairwise duplicate check is $\Theta(n^2)$, M2's doubling-window audit is $\Theta(n)$, S2 binary search is $\Theta(\log n)$, current M4 is $\Theta(n^2)$, redesigned M4 is $\Theta(n \log n)$, M5 is $\Theta(n \log^2 n)$, and the report generator is $\Theta(n^2)$.

At one million records, M1, current M4, and the report generator exceed the 600-second processing budget under the stated operation model. The recommended priorities are to replace pairwise duplicate detection with a hash-based or sort-based method, remove unnecessary recursive replication in M4, and redesign the quadratic report computation. M2 and S2 have favourable growth; M5 should be confirmed by real profiling.

## References


- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. (2014). Data Structures and Algorithms in Python. Wiley.
- Levitin, A. (2012). Introduction to the Design and Analysis of Algorithms (3rd ed.). Pearson.
- Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
- Course assessment brief: CSA06 – Design and Analysis of Algorithms, CO1 Assessment Tool 1, ShopSphere Scalability Audit.